# 1. Gauss Transforms and LU Factorization [15 marks]

In lecture, we showed how Gauss transforms can be used to compute the $A = LU$ or $PA = LU$ factorizations of $A \in \mathbb{R}^{n\times n}$. You will attempt to prove certain properties of these transforms in this question.

(a) Let $L_k$ be the Gauss transform of order $n$ used to annihilate elements in the $k$-th column during the $k$-th stage of Gaussian Elimination. Show that $L_k$ can be written compactly in matrix-vector notation as:

$$L_k = I - m_k e_k^T,$$

where $e_k$ is the $k$-th column of the identity matrix, and:

$$e_i^T m_k = 0, \quad i = 1, 2, \ldots, k.$$

<u>Solution</u>:
Let $L_k$ be the Gauss transform of order $n$ used to annihilate elements in the $k$-th column during the $k$-th stage of Gaussian Elimination.
Define $m_k$ as the multipliers used to annihilate elements in the $k$-th column during the $k$-th stage of elimination.
Specifically, $m_k = [0, \ldots, 0, m_{k+1}, \ldots, m_n]^T$.
Where $m_i = \frac{a_i}{a_k}$, $i = k+1, k+2, \ldots, n$.
Now, denote $e_k$ as the $k$-th column of the identity matrix.
Then, $e_k^T = [0, \ldots, 1, \ldots, 0]$, where the 1 is in the $k$-th position.
We have that

$$m_k e_k^T = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ m_{k+1} \\ \vdots \\ m_n \end{bmatrix} \begin{bmatrix} 0 & \ldots & 1 & \ldots & 0 \end{bmatrix} = \begin{bmatrix} 0 & \ldots & 0 & \ldots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \ldots & m_{k+1} & \ldots & 0 \\ \vdots & \ldots & \vdots & \ldots & \vdots \\ 0 & \ldots & m_n & \ldots & 0 \end{bmatrix}$$

Where the column containing the multipliers $m_{k+1}, \ldots, m_n$ is the $k$-th column.
Now subtract this from the identity matrix to get

$$I - m_k e_k^T = \begin{bmatrix} 1 & 0 & \ldots & 0 \\ 0 & 1 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & 1 \end{bmatrix} - \begin{bmatrix} 0 & \ldots & 0 & \ldots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \ldots & m_{k+1} & \ldots & 0 \\ \vdots & \ldots & \vdots & \ldots & \vdots \\ 0 & \ldots & m_n & \ldots & 0 \end{bmatrix} = \begin{bmatrix} 1 & \ldots & 0 & \ldots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \ldots & 1 & \ldots & 0 \\ \vdots & \ldots & -m_{k+1} & \ddots & \vdots \\ 0 & \ldots & \vdots & \ldots & 0 \\ 0 & \ldots & -m_n & \ldots & 1 \end{bmatrix}$$

This is exactly the form of the Gauss transform $L_k$ used to annihilate elements in the $k$-th column during the $k$-th stage of elimination.
Therefore, $L_k = I - m_k e_k^T$.

(b) Using the notation in (a), prove that $L_k^{-1}$ can be computed simply by changing the sign of each multiplier (i.e., by changing the sign of each non-zero element in $m_k$).
<u>Solution</u>:
Consider $I + m_k e_k^T$, where we now flip the signs of each multiplier.
We show that $I + m_k e_k^T = L_k^{-1}$.

Consider adding $m_k e_k^T$ to the identity matrix $I$:

$$I + m_k e_k^T = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} + \begin{bmatrix} 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \dots & m_{k+1} & \dots & 0 \\ \vdots & \dots & \vdots & \dots & \vdots \\ 0 & \dots & m_n & \dots & 0 \end{bmatrix} = \begin{bmatrix} 1 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & 0 \\ \vdots & \dots & m_{k+1} & \ddots & \vdots \\ 0 & \dots & \vdots & \dots & 0 \\ 0 & \dots & m_n & \dots & 1 \end{bmatrix}$$

Now consider $(I + m_k e_k^T)(L_k)$:

$$\begin{bmatrix} 1 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & 0 \\ \vdots & \dots & m_{k+1} & \ddots & \vdots \\ 0 & \dots & \vdots & \dots & 0 \\ 0 & \dots & m_n & \dots & 1 \end{bmatrix} \begin{bmatrix} 1 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & 0 \\ \vdots & \dots & -m_{k+1} & \ddots & \vdots \\ 0 & \dots & \vdots & \dots & 0 \\ 0 & \dots & -m_n & \dots & 1 \end{bmatrix} = \begin{bmatrix} 1 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & 0 \\ \vdots & \dots & m_{k+1} - m_{k+1} & \ddots & \vdots \\ 0 & \dots & \vdots & \dots & 0 \\ 0 & \dots & m_n - m_n & \dots & 1 \end{bmatrix} = I$$

Therefore, $L_k^{-1} = I + m_k e_k^T$.

(c) Using (a) and (b), prove that $(L_k L_j)^{-1}, j < k$, can be computed simply by adding the two transforms, changing the sign of each multiplier, and subtracting the identity.
<u>Solution</u>:
Consider $(L_k L_j)^{-1}$.
We know that $L_k = I - m_k e_k^T$ and $L_j = I - m_j e_j^T$.
We also know that $L_k^{-1} = I + m_k e_k^T$ and $L_j^{-1} = I + m_j e_j^T$.
Now consider $(L_k L_j)^{-1}$:

$$\begin{aligned} (L_k L_j)^{-1} &= (L_j)^{-1}(L_k)^{-1} \\ &= (I + m_j e_j^T)(I + m_k e_k^T) \\ &= I + m_j e_j^T + m_k e_k^T + m_j e_j^T m_k e_k^T \\ &= I + m_j e_j^T + m_k e_k^T + 0 && \text{Since } e_i^T m_k = 0 \text{ for } i < k, \text{ and we have } j < k \\ &= (I + m_j e_j^T) + (I + m_k e_k^T) - I \\ &= L_j^{-1} + L_k^{-1} - I \end{aligned}$$

Therefore $(L_k L_j)^{-1} = L_j^{-1} + L_k^{-1} - I$.

(d) Let $P_i \equiv P_{ij} \in \mathbb{R}^{n \times n}$ be the permutation matrix that, when pre-multiplied to another matrix, swaps rows $i$ and $j$ with $i < j$. Using the notation for $L_k$ in (a), prove that $\tilde{L}_k = P_i L_k P_i$ is just $L_k$ with multipliers $i$ and $j$ swapped, but only if $i > k$.
<u>Solution</u>:
Suppose we want to swap rows $i$ and $j$ of a matrix.
In order to make the permutation matrix $P_i$ that swaps rows $i$ and $j$, we start with the identity matrix and then move the 1 at position $(i, i)$ to position $(i, j)$ and the 1 at position $(j, j)$ to position $(j, i)$.

From part a), we that

$$L_k = \begin{bmatrix} 1 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & 0 \\ \vdots & \dots & -m_{k+1} & \ddots & \vdots \\ 0 & \dots & \vdots & \dots & 0 \\ 0 & \dots & -m_n & \dots & 1 \end{bmatrix}$$

Arbritarily consider the $i$-th and $j$-th multipliers to be $-m_{k+1}$ and $-m_n$.
Let's visualize what happens when we swap rows $i$ and $j$.
First let's write $L_k$ so we can clearly see the entry 1 on the $i$-th and $j$-th rows, which will be underlined.

$$L_k = \begin{bmatrix} 1 & \dots & 0 & \dots & \dots & 0 \\ \vdots & \ddots & \vdots & \dots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & \dots & 0 \\ \vdots & \dots & -m_{k+1} & \underline{1} & \dots & \vdots \\ 0 & \dots & \vdots & \dots & \ddots & 0 \\ 0 & \dots & -m_n & \dots & \dots & \underline{1} \end{bmatrix}$$

When we swap rows $i$ and $j$, we move the 1 on the $i$-th row to the $j$-th row and the 1 on the $j$-th row to the $i$-th row.

$$P_i L_k = \begin{bmatrix} 1 & \dots & 0 & \dots & \dots & 0 \\ \vdots & \ddots & \vdots & \dots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & \dots & 0 \\ 0 & \dots & -m_n & \dots & \dots & \underline{1} \\ 0 & \dots & \vdots & \dots & \dots & 0 \\ \vdots & \dots & -m_{k+1} & \underline{1} & \dots & \vdots \end{bmatrix}$$

Multiplying $P_i$ on the left is responsible for swapping the multipliers, but notice that the ones are now mispositioned.
However we know that multiplying $P_i$ on the right will swap columns $i$ and $j$.

$$P_i L_k P_i = \begin{bmatrix} 1 & \dots & 0 & \dots & \dots & 0 \\ \vdots & \ddots & \vdots & \dots & \ddots & \vdots \\ 0 & \dots & 1 & \dots & \dots & 0 \\ \vdots & \dots & -m_{k+1} & \underline{1} & \dots & \vdots \\ 0 & \dots & \vdots & \dots & \ddots & 0 \\ 0 & \dots & -m_n & \dots & \dots & \underline{1} \end{bmatrix} = L_k$$

Since $L_k$ is filled with zeroes besides the ones on the diagonal and multipliers beneath the $k$-th row, along with the given fact that $k < i < j$, it is clear that swapping rows $i$ and $j$ then swapping columns $i$ and $j$ will only swap the multipliers on the $i$-th and $j$-th rows, with the swapping of the ones of those rows cancelling out on the second matrix multiplication.
Therefore, $P_i L_k P_i = L_k$.

## 2. PA = LU Factorization [15 marks]

Consider the linear system $Ax = b$ where:

$$A = \begin{bmatrix} 2 & 5 & 10 \\ 8 & 32 & 8 \\ 1 & 8 & 13 \end{bmatrix}, \quad b = \begin{bmatrix} 7 \\ -16 \\ 6 \end{bmatrix}.$$

(a) Compute the $PA = LU$ factorization of $A$. Use exact arithmetic. Show all intermediate calculations, including Gauss transforms and permutation matrices.

<u>Solution</u>:

We pivot at each step before applying the Gaussian transform $L_i$.

$$P_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad P_1 A = \begin{bmatrix} 8 & 32 & 8 \\ 2 & 5 & 10 \\ 1 & 8 & 13 \end{bmatrix}$$

$$L_1 = \begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{4} & 1 & 0 \\ -\frac{1}{8} & 0 & 1 \end{bmatrix} \quad L_1 P_1 A = \begin{bmatrix} 8 & 32 & 8 \\ 0 & -3 & 8 \\ 0 & 4 & 12 \end{bmatrix}$$

$$P_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad P_2 L_1 P_1 A = \begin{bmatrix} 8 & 32 & 8 \\ 0 & 4 & 12 \\ 0 & -3 & 8 \end{bmatrix}$$

$$L_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{3}{4} & 1 \end{bmatrix} \quad L_2 P_2 L_1 P_1 A = \begin{bmatrix} 8 & 32 & 8 \\ 0 & 4 & 12 \\ 0 & 0 & 17 \end{bmatrix} = U$$

Consider $P_2 L_1 P_2$:

$$\begin{aligned} P_2 L_1 P_2 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{4} & 1 & 0 \\ -\frac{1}{8} & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{8} & 1 & 0 \\ -\frac{1}{4} & 0 & 1 \end{bmatrix} \end{aligned}$$

So now we have $L_2 P_2 L_1 P_1 A = L_2 P_2 L_1 P_2 P_2 P_1 A = L_2 \hat{L}_1 P_2 P_1 A$ (*) from above and using the fact that the inverse of any permutation matrix is itself.

Let $P = P_2 P_1 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$.

And let $L = \hat{L}_1^- 1 L_2^- 1 = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{8} & 1 & 0 \\ \frac{1}{4} & -\frac{3}{4} & 1 \end{bmatrix}$.

So now we have $L_2 P_2 L_1 P_1 A = L_2 \hat{L}_1 P_2 P_1 A = U$ from (*)

$\Rightarrow P_2 P_1 A = \hat{L}_1^- 1 L_2^- 1 U$

$\Rightarrow PA = LU = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{8} & 1 & 0 \\ \frac{1}{4} & -\frac{3}{4} & 1 \end{bmatrix} \begin{bmatrix} 8 & 32 & 8 \\ 0 & 4 & 12 \\ 0 & 0 & 17 \end{bmatrix}$

(b) Use the factorization computed in (a) to solve the system.
Continuing from above,
So $Ax = b \Rightarrow PAx = Pb \Rightarrow LUx = Pb$.
Now let $Ux = d$ and solve $Ld = Pb$.
First find $Pb$:

$$Pb = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 7 \\ -16 \\ 6 \end{bmatrix} = \begin{bmatrix} -16 \\ 6 \\ 7 \end{bmatrix}$$

Solving $Ld = Pb$ using /cases and forward substition:

$$\begin{cases} d_1 = -16 \Rightarrow d_1 = -16 \\ \frac{1}{8}d_1 + d_2 = 6 \Rightarrow \frac{1}{8}(-16) + d_2 = 6 \Rightarrow -2 + d_2 = 6 \Rightarrow d_2 = 8 \\ \frac{1}{4}d_1 - \frac{3}{4}d_2 + d_3 = 7 \Rightarrow \frac{1}{4}(-16) - \frac{3}{4}(8) + d_3 = 7 \Rightarrow -4 - 6 + d_3 = 7 \Rightarrow d_3 = 17 \end{cases}$$

So $d = \begin{bmatrix} -16 \\ 8 \\ 17 \end{bmatrix}$.
Now solve $Ux = d$ using a system of equations using /cases and back substition:

$$\begin{cases} 17x_3 = 17 \Rightarrow x_3 = 1 \\ 4x_2 + 12x_3 = 8 \Rightarrow 4x_2 + 12 = 8 \Rightarrow 4x_2 = -4 \Rightarrow x_2 = -1 \\ 8x_1 + 32x_2 + 8x_3 = -16 \Rightarrow 8x_1 - 32 + 8 = -16 \Rightarrow 8x_1 = 8 \Rightarrow x_1 = 1 \end{cases}$$

Therefore the solution to the system is $x = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$.

(c) Why is Gaussian Elimination usually implemented as in this question (i.e., $PA = LU$ is computed separately, and then the factorization is used to solve $Ax = b$)?
<u>Solution</u>:
Once we have the $LU$ factorization we can solve the system for any $b$ without having to recompute the factorization.
So for numerous potential right hand sides, we can solve the systems $Ld = b$ and $Ux = d$ in $O(n^2)$ time, compared to the $O(n^3)$ time it would take to recompute the factorization for each $b$, and even worse if we were solved $Ax = b$ directly, which would also take $O(n^3)$ time.
Therefore, it is more efficient to compute the $PA = LU$ factorization separately and then use it to solve $Ax = b$.

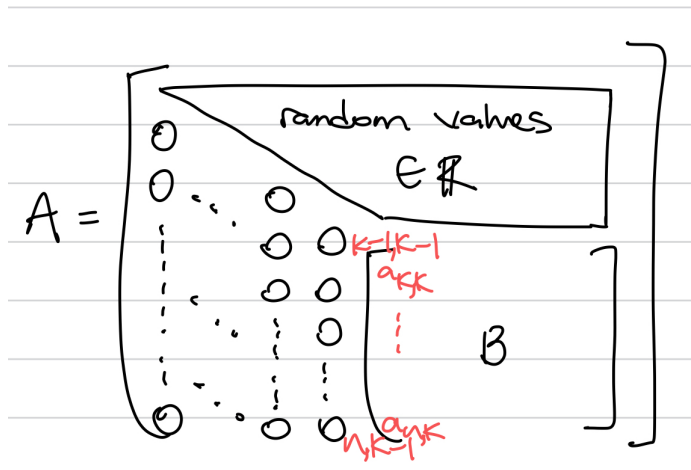
## 3. Full Pivoting and Stability [20 marks]

In lecture, we saw that Gaussian Elimination with row partial pivoting leads to a more stable factorization of $A \in \mathbb{R}^{n \times n}$. Full pivoting, which employs both row and column interchanges, guarantees a stable factorization.

(a) Sketch the general structure of an $n \times n$ matrix just before the $k$-th stage of the elimination, and show how row and column interchanges move the largest element in magnitude in the current matrix to the pivot position. Ensure previously annihilated elements from the first $k - 1$ stages are not reinstated.
<u>Solution</u>:
First let's denote the matrix $A$ as $A = \begin{bmatrix} a_{ij} \end{bmatrix}$ with dimension $n$.
At the $k$-th stage of elimination denote the submatrix $B$ as

$$B = \begin{bmatrix} a_{kk} & a_{k,k+1} & \cdots & a_{kn} \\ a_{k+1,k} & a_{k+1,k+1} & \cdots & a_{k+1,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,k} & a_{n,k+1} & \cdots & a_{nn} \end{bmatrix}$$

So $B$ only contains entries where the row and column indices are greater than or equal to $k$, allowing us to safely interchange rows and columns of $B$ without reinstating previously annihilated elements into entries outside of $B$. Here is a sektch of the full matrix $A$ containing $B$ at before the k-th stage of elimination.
:




Now let's describe the algorithm for full pivoting using pseudocode.
Below is the pseudocode for finding the largest element in magnitude in the submatrix matrix $B$.

```
initialize max_value=0, max_row=k, max_col=k
for row in range(k to n):
    for col in range(k to n):
        if abs(A[i][j]) > abs(max_value):
            set max_value = A[i][j]
            set max_row = row
            set max_col = col
```

After obtaining the largest element in magnitude, simply swap row $k$ with row $max_row$ and column $k$ with column $max_col$.
This will move the largest element in magnitude in the current matrix to the pivot position.

(b) Full pivoting leads to a $PAQ = LU$ factorization, where $P$ and $Q$ are permutation matrices. Show algebraically how this factorization can be used to solve $Ax = b$ with forward elimination and backsolve on a lower and upper triangular system, respectively. What is the effect of $Q$ on the original system?
<u>Solution</u>:
Given the $PAQ = LU$ factorization, we can solve $Ax = b$ as follows.
From $PAQ = LU$, we have $PAQQ = LUQ \Rightarrow PA = LUQ$ since permutation matrix $Q$ which swaps columns is its own inverse.
So given $Ax = b$, we have $PAx = Pb \Rightarrow LUQx = Pb$.
We can still first solve $Ld = b$ and then solve $UQx = d$.
To solve $UQx = d$, we can first apply the permutation $Q$ onto $U$ so let $O = UQ$.
Solving $Ox = d$ which will in fact still allow for back substition even though the upper triangular matrix $U$ has been permuted.
Therefore the effect of $Q$ on the original system is that it permutes the columns of the matrix $U$ during the back-solving portion to find $x$.

(c) Repeat question 2, but this time use full pivoting.
<u>Solution</u>:
We will use the same matrix $A$ as in question 2.

We find the largest element in magnitude in the submatrix $B$ at each stage of elimination.

$$P_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad P_1A = \begin{bmatrix} 8 & 32 & 8 \\ 2 & 5 & 10 \\ 1 & 8 & 13 \end{bmatrix} Q_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} P_1AQ_1 = \begin{bmatrix} 32 & 8 & 8 \\ 5 & 2 & 10 \\ 8 & 1 & 13 \end{bmatrix}$$

$$L_1 = \begin{bmatrix} 1 & 0 & 0 \\ -\frac{5}{32} & 1 & 0 \\ -\frac{1}{4} & 0 & 1 \end{bmatrix} \quad L_1P_1AQ_1 = \begin{bmatrix} 32 & 8 & 8 \\ 0 & 0.75 & 8.75 \\ 0 & -1 & 11 \end{bmatrix}$$

$$P_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad P_2L_1P_1AQ_1 = \begin{bmatrix} 32 & 8 & 8 \\ 0 & -1 & 11 \\ 0 & 0.75 & 8.75 \end{bmatrix} Q_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} P_2L_1P_1AQ_1Q_2 = \begin{bmatrix} 32 & 8 & 8 \\ 0 & 11 & -1 \\ 0 & 8.75 & 0.75 \end{bmatrix}$$

$$L_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{35}{44} & 1 \end{bmatrix} \quad L_2P_2L_1P_1AQ_1Q_2 = \begin{bmatrix} 32 & 8 & 8 \\ 0 & 11 & -1 \\ 0 & 0 & \frac{17}{11} \end{bmatrix}$$

Consider $P_2L_1P_2 = \hat{L}_1$:

$$\begin{aligned} P_2L_1P_2 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -\frac{5}{32} & 1 & 0 \\ -\frac{1}{4} & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{4} & 1 & 0 \\ -\frac{5}{32} & 0 & 1 \end{bmatrix} \end{aligned}$$

Consider $P_2P_1 = P$:

$$P_2P_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Consider $Q_2Q_1 = Q$:

$$Q_2Q_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Now let $L = \hat{L}_1^-1L_2^-1 = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{4} & 1 & 0 \\ \frac{5}{32} & \frac{35}{44} & 1 \end{bmatrix}$.

So now we have $L_2P_2L_1P_1AQ_1Q_2 = L_2P_2L_1P_2P_1AQ_1Q_2 = L_2\hat{L}_1PAQ = U \Rightarrow PAQ = LU$ from above.
Notice from $PAQ = LU$ we have $PA = LUQ$.
From $Ax = b$ we have now have $PAx = Pb \Rightarrow LUQx = Pb$.

Let $\hat{U} = UQ = \begin{bmatrix} 8 & 32 & 8 \\ -1 & 0 & 11 \\ \frac{17}{11} & 0 & 0 \end{bmatrix}$.

$$Pb = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 7 \\ -16 \\ 6 \end{bmatrix} = \begin{bmatrix} -16 \\ 6 \\ 7 \end{bmatrix}$$

We can still first solve $Ld = Pb$:

$$Ld = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{4} & 1 & 0 \\ \frac{5}{32} & \frac{35}{44} & 1 \end{bmatrix} \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} -16 \\ 6 \\ 7 \end{bmatrix}$$

So $d = \begin{bmatrix} -16 \\ 10 \\ \frac{17}{11} \end{bmatrix}$.

Now solve $\hat{U}x = d$ by first writing out their matrices like how we did for $Ld = b$:

$$\hat{U}x = \begin{bmatrix} 8 & 32 & 8 \\ -1 & 0 & 11 \\ \frac{17}{11} & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -16 \\ 10 \\ \frac{17}{11} \end{bmatrix}$$

So $x = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$ as wanted.

## 4. Solving Multiple Linear Systems [10 marks]

Let $A \in \mathbb{R}^{n \times n}$ be a nonsingular matrix, and let $x_i, t_i \in \mathbb{R}^n, i = 1, \ldots, k$.

(a) Explain how Gaussian Elimination with partial pivoting can be used to solve the $k$ linear systems:

$$Ax_1 = t_1, \quad Ax_2 = t_2, \ldots, Ax_k = t_k$$

as efficiently as possible. Derive the complexity of your approach by counting flops (multiplication/addition pairs). Include both $n$ and $k$ in your final operation count.

<u>Solution</u>:

We can employ Gaussian elimation with partial pivoting to obtain the $LU$ factorization once.

Since $A$ does not change, we only need to perform this step once.

When we apply the first Gaussian transform $L_1$ to $A$, we are performing $n-1$ multiplcation/addition pairs (FLOPS) for each entry in the $n-1, n-1$ submatrix of the resulting matrix product. Since there are $n-1$ entries in the submatrix, we have $(n-1)(n-1) = (n-1)^2$ FLOPS for the first stage of Gaussian elimation.

Likewise, for the second stage of Gaussian elimination, we have $(n-2)^2$ FLOPS, and so on.

In total, we have $\sum_{i=1}^{n-1} i^2 = \frac{n(n-1)(2n-1)}{6}$ FLOPS but we only perform $LU$ factorization once so we simply the number of FLOPs to be the highest term in the expression, $n^3/3$.

Now we can solve the $k$ linear systems $Ax_i = t_i$ by first solving $Ld_i = t_i$ for each $i$ and then solving $Ux_i = d_i$ for each $i$.

For each $i$, we have $n^2$ FLOPS for the forward substitution and $n^2$ FLOPS for the back substitution.

This is clear from the fact that in both forward/backward substition we start with 1 FLOP to determine the first entry, then 2 FLOPs to determine the second entry, and so on until we reach the $n$-th entry.

So expressing in summation notation, $\sum_{i=1}^{k} i = \frac{n(n-1)}{2}$ FLOPS for solving the $k$ linear systems.

Therefore, the total complexity is $\frac{n^3}{3} + k \cdot O(n^2)$.

(b) For $k = n$ and a suitable choice for the $n$ right-hand sides $t_1, t_2, \ldots, t_n$, your algorithm in (a) can be used to compute $A^{-1}$. Explain how this can be done.

Solution:

Let each $t_i$ be the $i$-th column of the identity matrix $I$.

So we have $Ax_i = e_i$ where $e_i$ is the $i$-th column of the identity matrix.

Therefore, we can solve the $n$ linear systems $Ax_i = e_i$ for each $i$ using the algorithm in (a).

After solving each system, we will have $n$ solutions $x_i$ where each $x_i$ is the $i$-th column of the inverse matrix $A^{-1}$.

We already know that the complexity of the algorithm in (a) is $\frac{n^3}{3} + k \cdot O(n^2)$.

Since $k = n$, the complexity is $\frac{n^3}{3} + n \cdot O(n^2)$.

Therefore the comlexity of this approach is $\frac{n^3}{3} + n \cdot O(n^2) = \frac{n^3}{3} + O(n^3)$.

(c) A possible scheme for solving $Ax = b$ is to first compute $A^{-1}$ using (b), and then compute the matrix-vector product $x = A^{-1}b$. Is this scheme preferable to the usual Gaussian Elimination algorithm with $PA = LU$ factorization, forward elimination, and backsolve? Provide operation counts to justify your answer.

Solution:

Generally finding the inverse first and then computing the matrix-vector product is not preferable to the usual Gaussian Elimination algorithm with $PA = LU$ factorization, forward elimination, and backsolve.

The process of computing the matrix-vector product is $n^2$ FLOPS for each entry in the resulting vector.

So the complexity of finding the inverse first and then computing the matrix-vector product is $\frac{n^3}{3} + O(n^3) + O(n^2)$.

Compared to the usual Gaussian Elimination algorithm with $PA = LU$ factorization, forward elimination, and backsolve,

$\frac{n^3}{3} + O(n^3) + O(n^2) \geq \frac{n^3}{3} + \cdot O(n^2)$.

Therefore, the usual Gaussian Elimination algorithm with $PA = LU$ factorization, forward elimination, and backsolve is preferable.

(d) Instead of using (b) to compute $A^{-1}$, we could use Cramer's rule. Would matrix inversion with Cramer's rule followed by the matrix-vector product $x = A^{-1}b$ be preferable to the usual Gaussian Elimination algorithm? Provide operation counts to justify your answer.

Solution:

Cramer's rule is not preferable to the usual Gaussian Elimination algorithm.

Cramer's rule involves computing the determinant of $A$ which is $O(n!)$ FLOPS.

However with our $LU$ factorization found in part a) we can find the determinant of $A$ in $O(n^3)$ FLOPS by finding the determinant of $L$ and $U$ and multiplying them together.

Since $L$ and $U$ are both triangular matrices, the determinant of each are the product of the diagonal entries.

So the complexity of finding the determinant of $A$ is $O(n^3)$.

Now we need to find the determinants of $n$ matrices $A_i$ where $A_i$ is the matrix obtained by replacing the $i$-th column of $A$ with $b$.

The complexity of finding the determinant of each $A_i$ is $O(n^3)$, so the total complexity is $n \cdot O(n^3) = O(n^4)$.

Therefore, matrix inversion with Cramer's rule followed by the matrix-vector product $x = A^{-1}b$ is not preferable to the usual Gaussian Elimination algorithm.